

CODE SPORT



Sandya Mannarswamy

In this month's column, we discuss the topic of testing natural language processing (NLP) applications.

Over the last couple of months, we have been discussing how to perform intent extraction in conversational systems. In this month's column, we take a break from this topic and cover an important but often ignored aspect of natural language processing — testing of NLP applications. A few of our readers, in fact, reached out to me with questions on this topic. Hopefully, this column and the subsequent ones will answer their questions.

Most of our readers will be familiar with testing traditional software applications. For example, let us assume that you have written a simple application that takes in an employee's name, and returns his salary details by looking this up from the back-end. How would you go about testing this? The first task is to make sure that your application works as intended. This is known as correctness or functionality testing.

Given a set of known (input, output) combinations of employees' names and salary details, the tests will check whether each time the application produced the correct salary details. This is known as oracle testing, where there is a test oracle that knows the right answer for each and every input combination and hence, given the input, the output from the application is compared with the ground truth from the test oracle. If they match, the test is declared as passed. Else it is declared as failed.

Once correctness testing is done, you may also want to do testing to cover the input space of your application. What would happen if the user provides an input that is not expected by the application? Does the application still behave correctly? For instance, our application takes as input an employee's name. We have been intentionally vague so far when defining the criteria for our input.

Employee names can only consist of a set of one or more words, with each word consisting of letters of the alphabet. Now, how does our application behave when the employee's name consists of all numbers, a mix of alphanumeric characters, a mix of alphanumeric and special characters, or single letters/digits? Given the constraints on the input, these can be defined as error conditions with illegal input values, and the application should be able to gracefully handle the error arising out of these illegal input values.

One important thing to keep in mind when we take user input for an application is that the input can be malformed either intentionally or unintentionally. That's why it is necessary to validate the input before it gets used by the application code. For instance, let us assume that you have coded your application in C programming language. You naively assumed that the name input for your application will be of a length less than 20 characters. So you defined a character array of length 20, to hold the name. You decided to use the input function 'scanf' to read in the input string to the 'name' array and then used a 'strcpy' to pass it into another array for validation code. You could have very easily exposed your application to a vulnerability known as a buffer overflow attack, whereby users can pass in unintended inputs. This could end up with a malicious user gaining control of your application.

The moral of the story is that one should not assume that the input to your application will be as you intended to be. Illegal, unexpected or erroneous inputs can be passed onto your application by end users either intentionally or unintentionally. It is your responsibility as the application's author to ensure that your application (a) recognises that it has